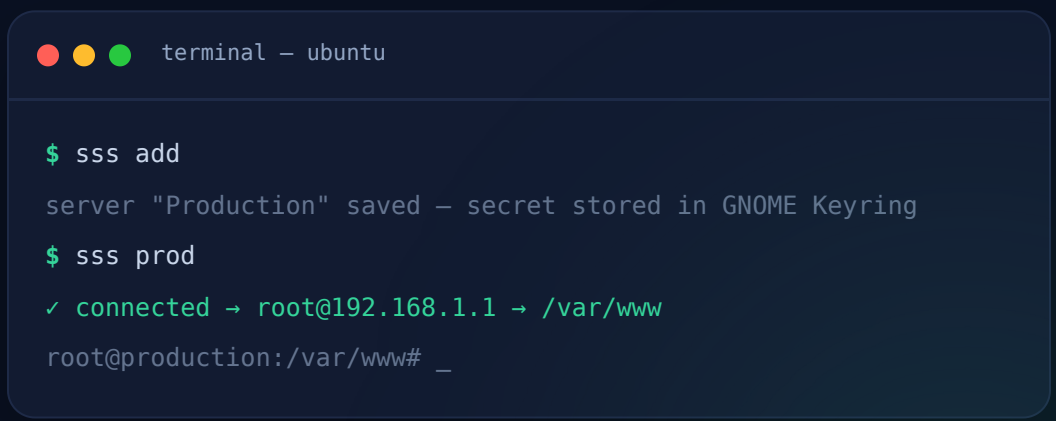
SSH Desk · Version 1.0 · User Guide

Professional SSH connection manager for Ubuntu

User Guide

SSH Desk

From registering a server to a real interactive shell — with a single command. Desktop GUI, command line and one shared core; passwords live only in GNOME Keyring, and the SSH engine is your system's own OpenSSH.



Real PTY terminal

Secrets only in the Keyring

SFTP file manager

Local/Remote/SOCKS tunnels

Offline install with mirrors

Edition 1.0 — November 2026

github.com/hmplus28/SSH-Desk

Contents

In this guide

01 Introducing SSH Desk

02 Installation & setup

03 Quick start: your first server

04 Connecting & managing paths

05 Complete sss CLI reference

06 The desktop GUI

07 The real terminal (PTY)

08 File management with SFTP

09 Keys, tunnels & commands

10 The security model

11 Backup & troubleshooting

Chapter 01

Introducing SSH Desk

SSH Desk is a real, installable product for managing all of your SSH servers on Ubuntu — not a demo. Three modules — the desktop GUI, the command line and the shared

core — work on exactly one database and one secret store: register a server in the GUI and connect to it from any terminal with `sss prod` in the same moment, even when the GUI is not running at all.

The connection engine is your **system's own OpenSSH**; the SSH protocol is not re-implemented, so connection behavior is exactly like Ubuntu itself. The built-in terminal is a **real PTY**: colors, arrow keys, **Ctrl+C**, vim and nano all work correctly. Passwords are stored only in **GNOME Keyring** and never reach the database, logs, command arguments or environment variables.

GUI — desktop interface

PySide6 with dark/light themes, 7 UI languages with full RTL for Persian and Arabic, multi-tab terminal and file manager.

CLI — command line

The `sss` command works without the GUI; the command name is configurable; bash/zsh completion included.

Core — shared engine

SQLite + SQLAlchemy with migrations, the Keyring service, the ssh command builder and the Host Key policy.

3	9	30+
modules, one engine	pages in the GUI	CLI subcommands

Chapter 02

Installation & setup

› Prerequisites

```
# Ubuntu 22.04 / 24.04+
sudo apt update
sudo apt install -y openssh-client gnome-keyring libsecret-1-0 \
     python3 python3-venv python3-pip
```

› Option A — user-level install (recommended, no sudo)

```
git clone https://github.com/hmplus28/SSH-Desk.git
cd SSH-Desk
bash scripts/install.sh
```

The script checks Python and OpenSSH, creates a virtual environment in `~/.local/share/ssh-desk/venv`, installs the package with all dependencies and creates the `ssh-desk` (GUI), `ssh-desk-cli`, `ssh-desk-askpass` and `sss` shortcuts in `~/.local/bin`; bash completion and the application icon are installed too.

› Option B — system-wide DEB package

```
bash scripts/package_deb.sh
sudo apt install ./dist/ssh-desk_1.0.0_amd64.deb
```

The DEB installs a self-contained PyInstaller bundle under `/opt/ssh-desk` and registers the menu entry and system icon. Upgrading is done by installing the newer package in place; your data and secrets remain untouched.

› Weak or interrupted internet: mirrors & offline install

If your network is unstable or PyPI is hard to reach, don't worry: every pip call in the install scripts automatically tries a **chain of mirrors** — if one mirror is unreachable it is skipped without any action from you and the next one is tried. The default order is **official PyPI → ArvanCloud (Iran) → TUNA → Aliyun → Tencent**.

```
# force a specific working mirror
bash scripts/install.sh --mirror-name ir    # ArvanCloud (Iran)
bash scripts/install.sh --mirror-name tuna  # TUNA (China)

# 1) download everything once (on any working network)
bash scripts/download_wheels.sh --mirror-name ir
# 2) install later with NO network at all
bash scripts/install.sh --offline
```

In `--offline` mode, pip is not allowed any network connection and installs only from the `vendor/wheels` folder — so an internet outage in the middle of setup can no longer leave the install half-finished. The same mirror chain is used by `scripts/build.sh`, so the DEB build is protected from outages as well.

› Where are the important files?

Location	Path
Application database	`~/.local/share/ssh-desk/ssh-desk.db`
Settings	`~/.config/ssh-desk/config.toml`
App known_hosts	`~/.local/share/ssh-desk/known_hosts`
Generated keys	`~/.local/share/ssh-desk/keys/`
Logs	`~/.local/share/ssh-desk/logs/`
Passwords & secrets	GNOME Keyring (Secret Service)

Uninstalling: `bash scripts/uninstall.sh` explicitly asks before removing data and secrets; without your confirmation nothing is deleted.

Chapter 03

Quick start: your first server

Step 1 — launch: run `ssh-desk` to open the GUI; or stay entirely in the terminal and use `sss add`.

Step 2 — add a server: on the Servers page press “Add server”. Enter the name, the alias (unique — this is what you connect with), host, port, username and default path. If you choose password auth, the secret is stored in GNOME Keyring immediately — not in the database. Before saving, the **Test** button performs a real connection so you know everything is correct.

```
# the same thing from the terminal (interactive wizard)
sss add
sss add --alias prod --host 192.168.1.1 --port 22 --user root \
        --auth password --path /var/www
```

Step 3 — connect: from any terminal type `sss prod` to open an interactive shell at the default path. In the GUI, double-clicking a server is enough to open a real terminal tab.

First connection: the host key fingerprint is shown; compare it with your server administrator and accept it once. If the host key ever **changes**, the connection is rejected immediately and guidance for checking a possible MITM attack is displayed.

Chapter 04

Connecting & managing paths

Every server can have a **default path** plus any number of **named saved paths**; for example define `website → /var/www/html` and `nginx → /etc/nginx` once for the production box and always jump to that folder with a single word:

```
sss prod                # shell at the server's default path
sss prod nginx          # shell at the saved path named "nginx"
sss prod /var/www       # shell at any absolute path
sss prod --exec "uptime"           # run one command, no shell
sss prod --user ubuntu --port 2222 # per-run override (NOT saved)
sss prod --dry-run                 # print the exact ssh command
```

If a requested path does not exist on the server, the error message is clear and understandable — not a vague disconnect. Paths are transferred with safe quoting (shlex) so spaces or special characters in folder names cause no trouble. The `--user` and `--port` options apply only to that run and are never saved.

› CLI exit codes

Code	Meaning
`0`	Operation succeeded
`1`	Runtime error (connection, network, Keyring, …)
`2`	Invalid input or wrong command usage
`130`	Interrupted with Ctrl+C

These codes make `sss` reliable inside your scripts and automations; for example you can check in CI or cron that a connection succeeded.

Chapter 05

Complete sss CLI reference

The `sss` command name is an example; rename it with `sss config cli-name cnc` to anything you like (cnc, sshgo, …) — the change applies system-wide (a new shortcut in `~/.local/bin`). Any first token that is not a known subcommand is interpreted as a server alias, so `sss prod` is the same as `sss connect prod`.

› Servers

Command	What it does
`sss list`	Table of servers; filter with `--group --favorites --search`
`sss add`	Interactive add; every field can also be passed directly
`sss info prod`	Full details; `--show-secret` requires typing the alias
`sss edit prod`	Edit fields; `--set-password` stores a new password
`sss delete prod`	Delete with confirmation; the Keyring secret is purged too
`sss search TERM`	Search name, alias, host, user and description

› Monitoring & configuration

Command	What it does
`sss history`	Connection history with server/limit/status filters
`sss current`	Active SSH sessions and running tunnels
`sss config list`	All settings and file locations
`sss config set KEY VALUE`	Change a setting
`sss config cli-name cnc`	Rename the command system-wide
`sss config completions`	Install bash/zsh completion

› Keys, tunnels & configuration transfer

Command	What it does
`sss keys generate`	Generate Ed25519/RSA/ECDSA in OpenSSH format
`sss keys import`	Register an existing key (with `--copy` into app storage at 0600)
`sss keys export-pub`	Export the public key
`sss tunnel start|stop`	Run a tunnel as a daemon detached from the terminal
`sss import --file ~/.ssh/config`	Import ssh config with preview and duplicate detection
`sss export FILE.toml`	Backup; secrets only with `--include-secrets`
`sss restore FILE.toml`	Restore a backup
`sss install-cli`	Rebuild the terminal shortcuts

Completion: bash completion is installed automatically (subcommands + server aliases). For zsh: `sss config completions --shell zsh --install`, then add the completions path to `~/.zshrc`.

Chapter 06

The desktop GUI

The main window has a collapsible sidebar with 9 pages and a status bar (active sessions and tunnels). The interface ships in 7 languages — English, Persian, Arabic, Chinese, Russian, Spanish and French — with true right-to-left layout for Persian and Arabic; the dark/light theme is switched in Settings.

Page	What it does
Dashboard	Overview of servers, active sessions and tunnels
Servers	Search, group/tag/favorite filters, duplicate, quick test
Add/Edit Server	Four tabs: general, authentication, connection & paths, advanced (jump host and ssh options)
Server Details	Details; the password is masked and revealed only after typing the alias
Terminal	Multi-tab real PTY sessions
Files	Two-pane SFTP file manager with server-to-server transfer
SSH Keys / Port Forwarding / Commands / History	Manage keys, tunnels, saved commands and connection history
Settings	CLI command name, theme, language, defaults, Keyring status, backup and ssh-config import

› Keyboard shortcuts

`Ctrl+1..9` switch pages · `Ctrl+N` new server · `Ctrl+K` new terminal session · `Ctrl+Q` quit.

> **GUI and CLI are always in sync:** both interfaces share one database, one Keyring and one settings store; a server created in the GUI is immediately known to `sss` in every terminal — and vice versa.

Chapter 07

The real terminal (PTY)

Every terminal tab is a real PTY session over OpenSSH — not a text box. Anything possible in a real terminal is possible here:

- Colors, arrow keys, Tab completion, and running **vim**, **nano** and **htop** work correctly.
- **Ctrl+C** and **Ctrl+D** interrupt and close processes exactly like a system terminal.

- **Copy:** select text with the mouse (auto-copy) or use the toolbar; **Paste:** middle-click, the toolbar or `Ctrl+V` inside the terminal.
- **Resizing:** dragging the window instantly informs the server-side program (SIGWINCH) — editors and htop redraw correctly.
- Each tab shows the server name and the server-side window title; to reconnect just open a new session for the same server.

> **Privacy:** terminal content is never logged; only connection status and duration are recorded in history.

Chapter 08

File management with SFTP

On the Files page pick a server and press Connect: the left pane is your local system and the right pane is the server. Navigate by double-clicking folders, the “up” button or editing the path directly. All everyday operations are supported:

- **Upload and download** via drag & drop or the buttons.
- **New folder, rename, delete** (with confirmation, recursive).
- **Change permissions** in octal form (e.g. 0644) — no mental math.
- **Edit text files:** the file opens locally, and after saving a re-upload is offered.
- **“Send to server…”:** move a file directly between two registered servers — without downloading through your machine.

> **If SFTP fails but the terminal works:** the server may have disabled the SFTP subsystem (`Subsystem sftp` in sshd_config) or the host key is not accepted yet — connect once via the terminal or press Test.

Chapter 09

Keys, tunnels & saved commands

› SSH keys

Generate an Ed25519 (recommended), RSA or ECDSA key in the standard OpenSSH format; the optional key passphrase is stored in the Keyring too. Register existing keys (without copying the file) or, by your explicit choice, copy them into app storage with 0600 permissions. The fingerprint column helps verification, the “linked servers” column shows consumers of each key, and “Install on server” builds the `ssh-copy-id` command and runs it in a terminal session.

```
sss keys generate --name deploy --type ed25519
sss keys import --path ~/.ssh/id_ed25519 --name personal
sss keys export-pub deploy
```

› Port tunnels

Three tunnel types: **Local** (e.g. `127.0.0.1:5432 → db-internal:5432` to reach an internal database), **Remote** (reverse — expose a local service port on the server) and **Dynamic** (a SOCKS5 proxy such as `127.0.0.1:1080`). Tunnels run in an independent daemon — they survive closing the GUI or CLI and their state is visible in `sss current`.

```
sss tunnel db       # start the tunnel named "db"
sss tunnel stop db
sss current         # live sessions + running tunnels
```

› Saved commands

Save frequently used commands per server or globally (name, command, category). Sensitive commands with the “requires confirmation” flag ask before running. Execute from the Commands page in the GUI or with `sss prod --exec "<command>"` in the terminal.

Chapter 10

The security model

SSH Desk is designed to hold the credentials of production servers and is built on one principle from start to finish: **secrets live only in the Secret Service**. The most important controls:

1 — Passwords only in GNOME Keyring

The database stores only a secret-less reference (JSON); the password itself never reaches SQLite (verified by automated tests).

2 — Secure askpass, not sshpass

OpenSSH runs with `SSH_ASKPASS_REQUIRE=force` and a secure helper that reads the secret "by reference" from the Keyring; the password never appears in argv, environment, logs or temp files.

3 — Host keys always strict

`StrictHostKeyChecking=yes` is final in the command builder and cannot be turned off; unknown host → fingerprint dialog; changed key → MITM warning and `ssh-keygen -R` guidance.

4 — Injection-resistant

All subprocess calls use argument arrays without shell=True; remote paths are quoted with shlex.quote and all input is validated with Pydantic.

› Other controls

- File permissions: database and config 0600, data folders 0700, private keys 0600.
- The logger scrubs `password=`, `secret=` patterns and secrets registered at runtime.
- Revealing/copying a password requires typing the server alias; the clipboard auto-clears after 30 seconds.
- Sensitive operations — deleting a server, recursive file deletion, full data purge (typing PURGE) — always ask for confirmation.
- No daemon runs as root; the GUI and CLI never need sudo.

- Key generation is fully in-process; the key passphrase never reaches argv or temp files.
- Export/backup is secret-free by default; including secrets is opt-in with an explicit warning.

Chapter 11

Backup, transfer & troubleshooting

› Backup & restore

```
sss backup backup.toml                  # no secrets by default
sss restore backup.toml
sss export full.toml --include-secrets  # opt-in + warning
sss import --file ~/.ssh/config         # preview, then: --apply
```

Import from `~/.ssh/config` happens with a preview and duplicate-server detection; the TOML export is for moving settings to another machine and contains no secrets until you explicitly ask.

› Common errors and fixes

Symptom	Solution
`sss: command not found`	Add `~/.local/bin` to PATH or run `sss install-cli`
“Secret Service is not available”	`sudo apt install gnome-keyring libsecret-1-0` and log out/in once
Keyring is locked	Unlock it once; the app also asks interactively when possible
Host key changed	Only after verification: `ssh-keygen -R "[host]:port"`, then accept again
Permission denied	Password: `sss edit prod --set-password` · Key: `sss keys list`
GUI does not open	`ssh-desk 2>&1 | tail -20`; install Qt libraries; Wayland: `QT_QPA_PLATFORM=xcb`
Tunnel port busy	`ss -tlnp | grep <port>`, then `sss tunnel stop`

Full reset: `sss uninstall-data` — you must type PURGE to wipe data and Keyring secrets.

Secure connections, always within reach.

SSH Desk is open source under the MIT license. If this guide helped you, visit the GitHub repository to report issues or suggest features.

`github.com/hmplus28/SSH-Desk`

MIT License